

QCon
全球软件开发大会

复杂业务场景下RCA Agent的探索实践

郭勇良

快手 资深服务端架构师



极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 背景和痛点

02 业务场景的落地挑战

03 核心机制和架构设计

04 总结和展望

01

背景和痛点

要解决的问题

AI浪潮之下..

编码工作大体上已被攻克

**“Coding is
largely
solved”**

Boris Chemy
Head of Claude Code



软件工程被解决了吗？



为什么需要RCA Agent

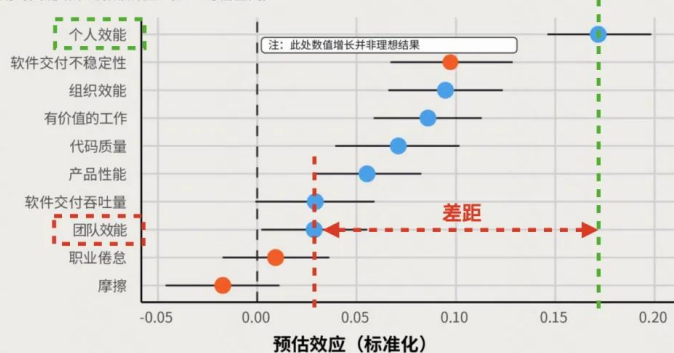
AI Coding是不够的
“排障是下个待解问题”

AI Coding后，组织效能 和 个人效能 间仍存在巨大差异

图 28 展示了 AI 采纳与这些结果之间的关系。²³

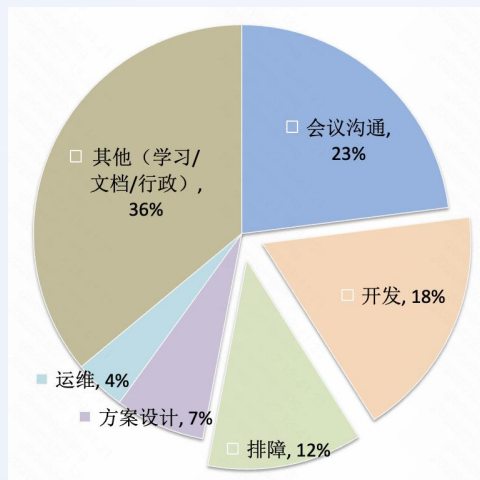
人工智能影响的格局演变

人工智能应用对关键结果的预估效应 (89%可信区间)



对于橙色标注的结果 (如职业倦怠)，负向效应才是理想的。

大型公司 (微软) + 成熟团队的
“典型工作日”分配



《2025年DORA报告：人工智能辅助软件开发现状调查报告》
微软《Today was a Good Day: The Daily Life of Software Developers》
5971 份有效响应 (响应率 15.5%) 平均从业年限 10 年。

随着人对代码掌控下降，
“AI排障从可选项变为必选项”

OpenClaw在一场史上最大版本的更新后，因激进、无兼容层的破坏性重构，大量用户反馈插件瘫痪、功能失效。

r/openclaw · 5d ago
Monobert Member

Do not install 2026.3.22 - package issues

Bug Report

Just updated to 2026.3.22 - Both whatsapp and the control UI are broken.

Bug reports indicate this is a broken package. If you have already upgraded there is a workaround for Control UI here:

<https://github.com/openclaw/openclaw/issues/52808> (Follow the instructions in the Chinese post)

For whatsapp it's being reported but no solution yet:

<https://github.com/openclaw/openclaw/issues/52813>

我们要解决什么场景？

问题场景….

基础设施层

 容器故障

 节点故障

 网络故障

中间件层

 Cache异常

 DB异常

 MQ异常

业务层

 核心指标下跌

 风暴告警

 跨系统传播

**RCA
Agent**

业务层问题特点

业务层异常是 **用户体验、收入** 的直接体现

业务逻辑可能随时调整，是**易变的**

业务问题是**"无法预测步骤数量"**的开放问题

同样的“时长下跌”问题：

- 可能是Redis慢查询、GC、基础设施异常、代码bug
- 每种根因的排查路径截然不同且无法预先枚举。
- 这种情况下，需要LLM动态决策下一步做什么（继续追踪日志？查找变更、回滚版本？）

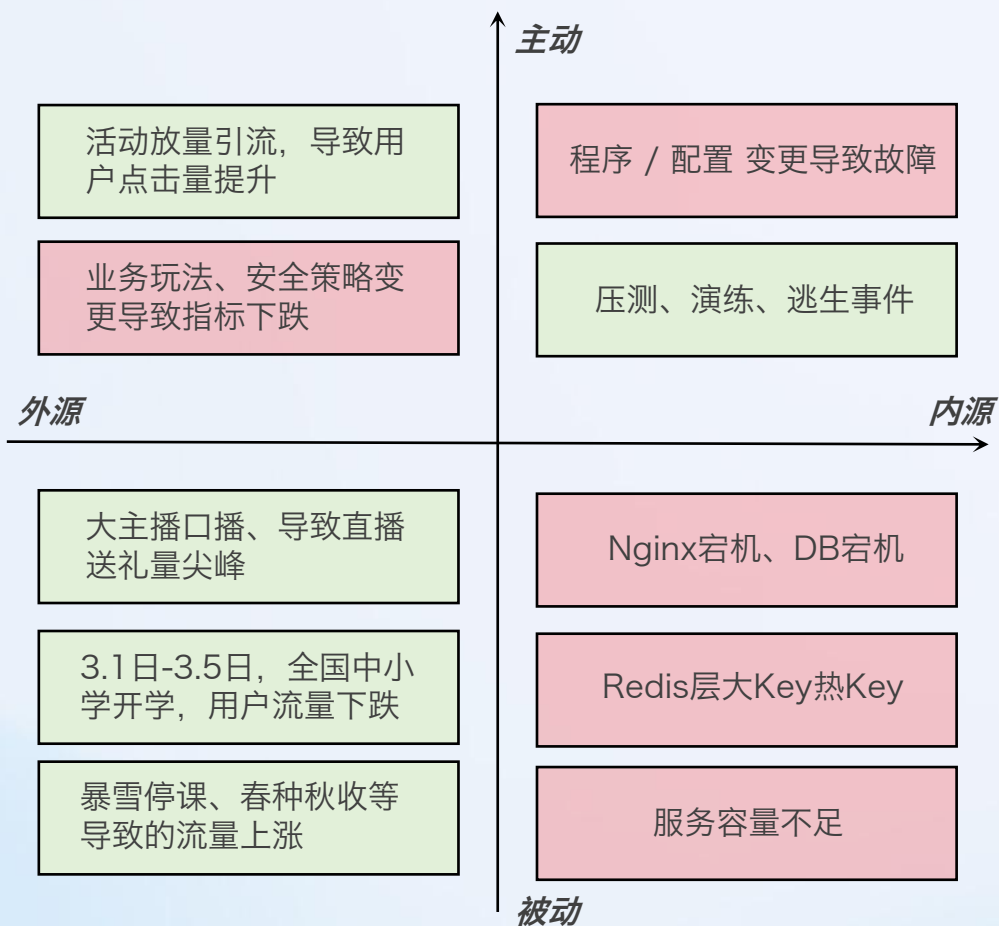
02

业务场景落地挑战

四个挑战：理解业务、对抗噪声、衡量不确定性、对抗幻觉

挑战一：如何让AI理解业务

可能引起告警或指标波动的事件



业务现实十分复杂

主动&被动 / 内部&外部 并存:

外部原因有相当一部分占比, 这部分难以归因。

噪声和信号并存:

很多告警是噪声、但也有非常重要的“信号”。

并非都是故障:

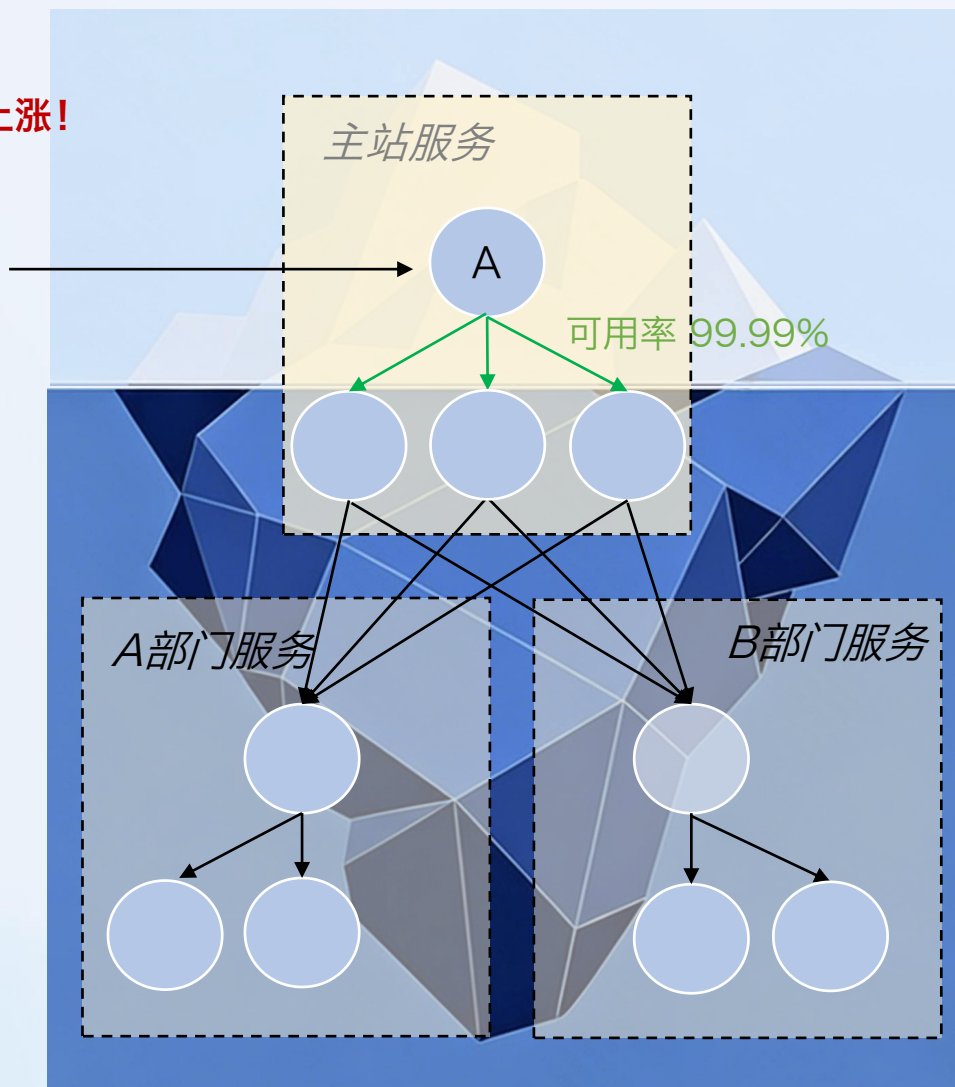
很多“信号”是预期内的、少部分预期外外引发故障。

巨大的状态空间:

排查内部链路指标之外, 也要考虑外部外部因素。

理解业务：业务场景案例 —— 请求异常上涨

！用户请求异常上涨！



A服务是核心业务入口服务，指标异常。

直接下游有100+，包含多部门服务，成功率全部正常。

首先，是不是问题？

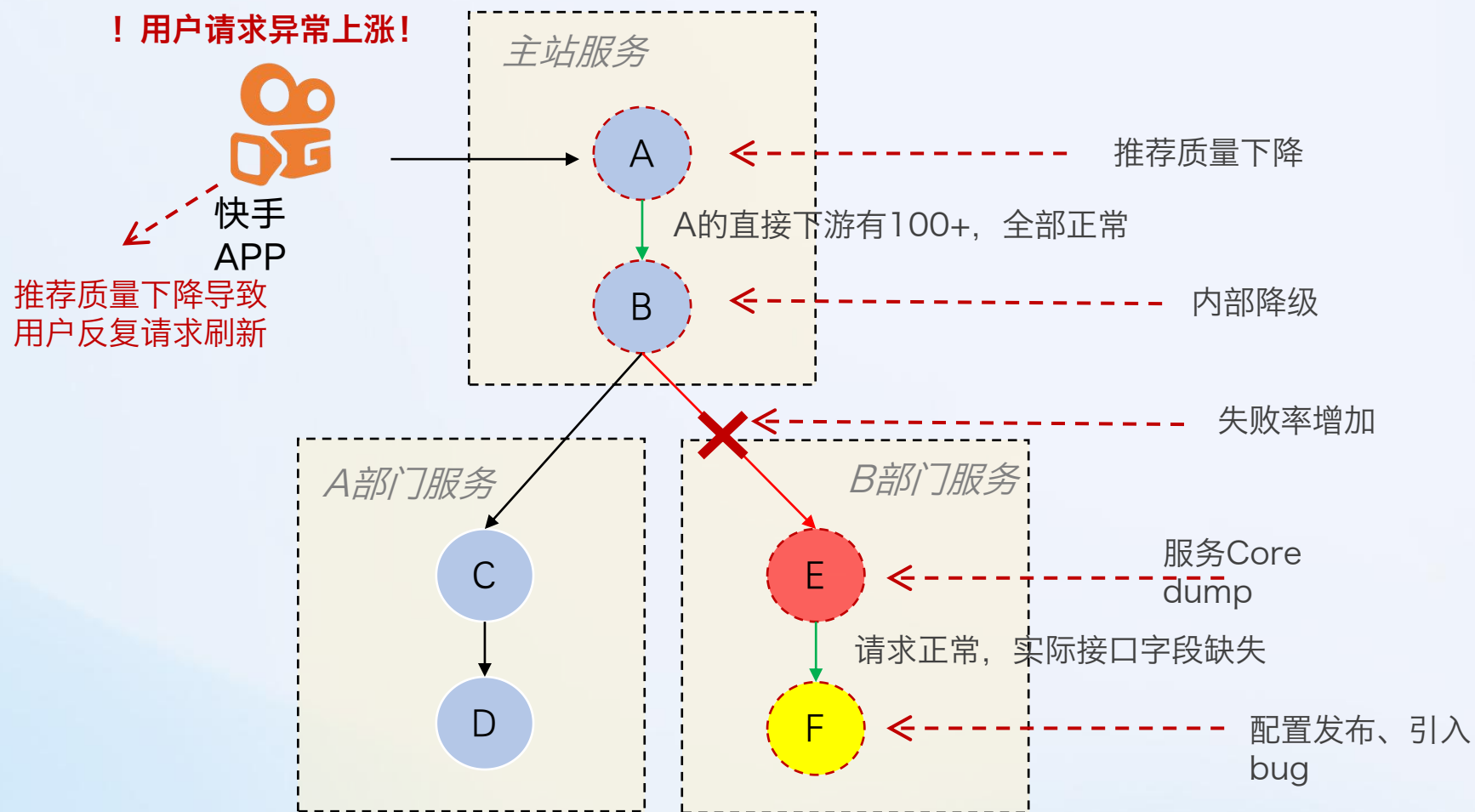
- 是外部有热点事件发生，用户涌入？
- 还是内部有啥变更影响了？

其次，假设是问题怎么排查？

- 拉所有下游owner逐一确认？
- 变更列表中所有可疑变更全部回滚？
- 扩大到业务域范围上升？



理解业务：业务场景案例 —— 真实根因



反常识：

推荐质量下降反而导致了请求量的增加。

异常传播中断：

中间服务降级、导致故障链路被“掩盖”，传导中断。

跨部门协同：

即使机制完善、效率再高，也有信息传递成本和折损。

一次排障，大量人员参与….

理解业务：业务场景案例 —— AI怎么解？

！用户请求异常上涨！



快手
APP

主站服务

A

A的直接下游有100+, 全部正常

推荐质量下降

B

内部降级

断点1:

Metric无法关联,
依赖业务经验

A部门服务

C

D

B部门服务

E

请求正常, 实际接口字段缺失

服务Core
dump

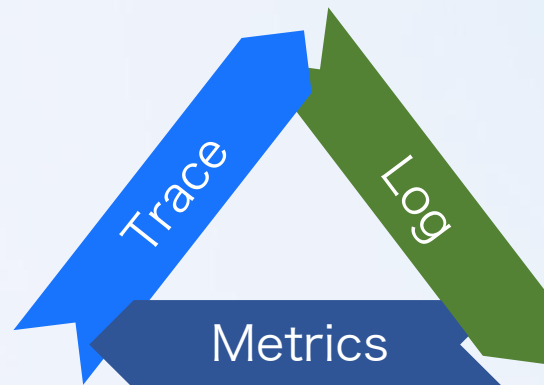
F

配置发布、引入bug

失败率增加

断点2:

Metric正常
业务Log缺失

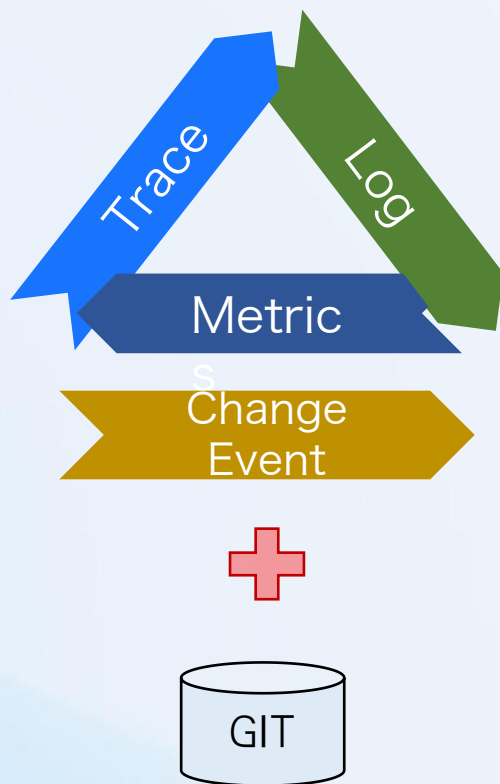


三板斧是否足够？

技术指标外，必须理解业务

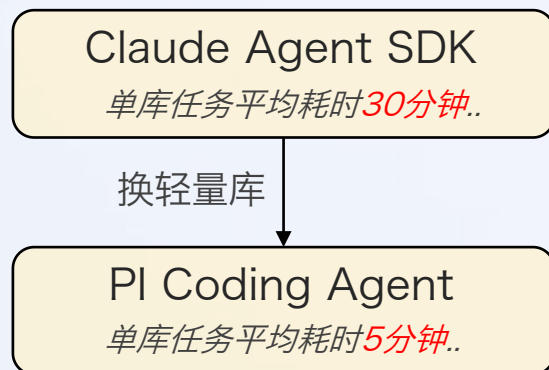
理解业务：建立业务上下文

通过读取链路相关代码，建立业务理解

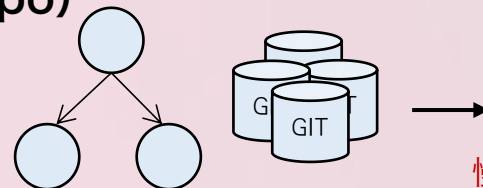


代码是唯一真实的文档
如：某业务错误码语义、
开关变更的逻辑影响

Coding Agent实时代码分析：



新的效率瓶颈：多库关联分析（3-5个Repo）



慢、累计耗时长

链路服务多、SDK依赖复杂、一次排障涉及多个仓库

$5\text{min} * (3 - 5) = 15\text{min} \sim 25\text{min}+$
累计耗时依然无法满足实时分析需求

理解业务：建立业务上下文

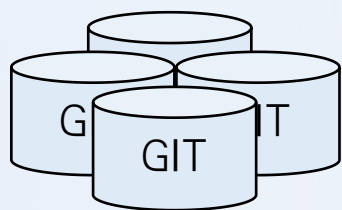
低抽象导致的低效率

```
31 def __init__(self, settings):
32     self.file = None
33     self.fingerprints = set()
34     self.logdups = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, "requests.log"),
39                         "a")
40         self.file.seek(0)
41         self.fingerprints.update([x.request() for x in self.requests])
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool("DEBUG", False)
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```

代码虽然真实，但是抽象层次极低

人也记不住每一行代码…

理解业务：建立业务上下文



业务代码

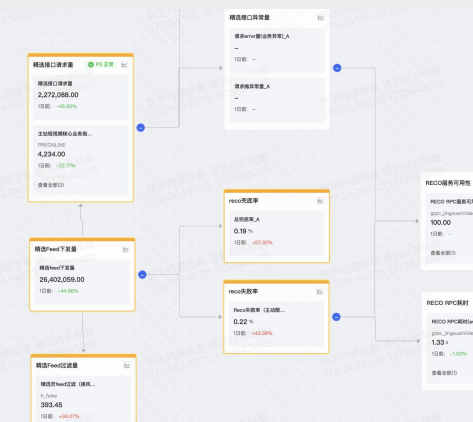
抽象



1.业务语义标准库

Code value (eg. 25667) → 业务Label (eg. 用户未授权)
提取错误码、metrics标注含义

2.指标拓扑图谱



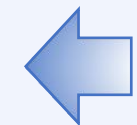
3.开关变更影响地图

Feature Flag



建立开关对接口、上下游关系业务影响

业务资产

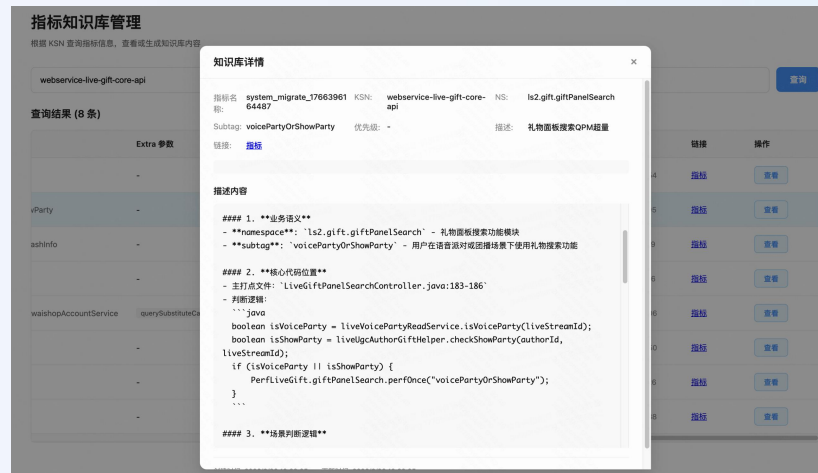


离线生成

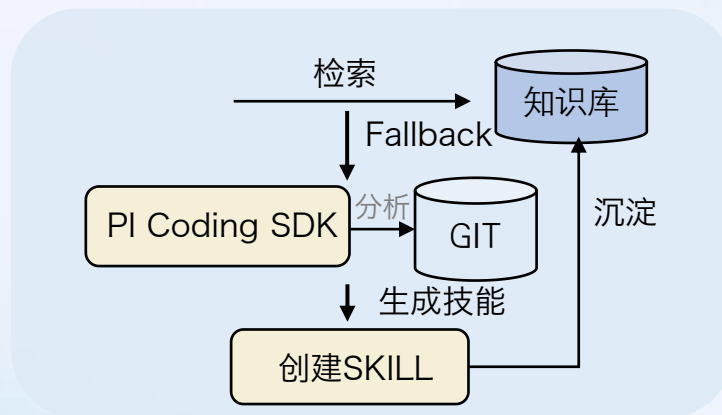


在线生成

离线沉淀：通过coding agent生成关系



在线沉淀：无业务资产时自动生成SKILL



理解业务：总结



本质是消除人和AI间的Context代差

Metrics

Trace

Log

Change Event

Agent

信息、信息、信息

RD

代码逻辑

错误码28801表示”需升级”；
开关A可能影响xx下游；

指标关系

评论量 关联 推荐下发量
关联 兜底触发量等；

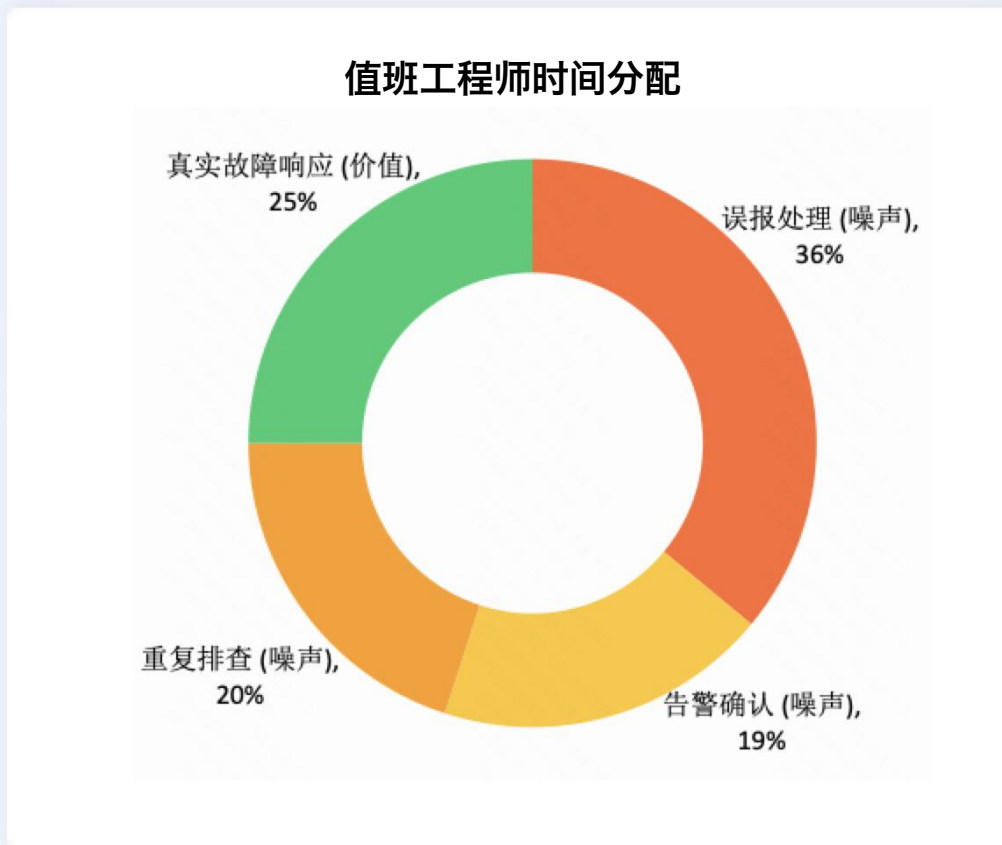
业务常识

大主播口播可能导致流量上涨；
中小学开学可能让请求下降

运维事件/外部事件

今天下游有压测/演练，流量上升；
故障注入可能导致错误率上升。

挑战二：如何对抗噪声

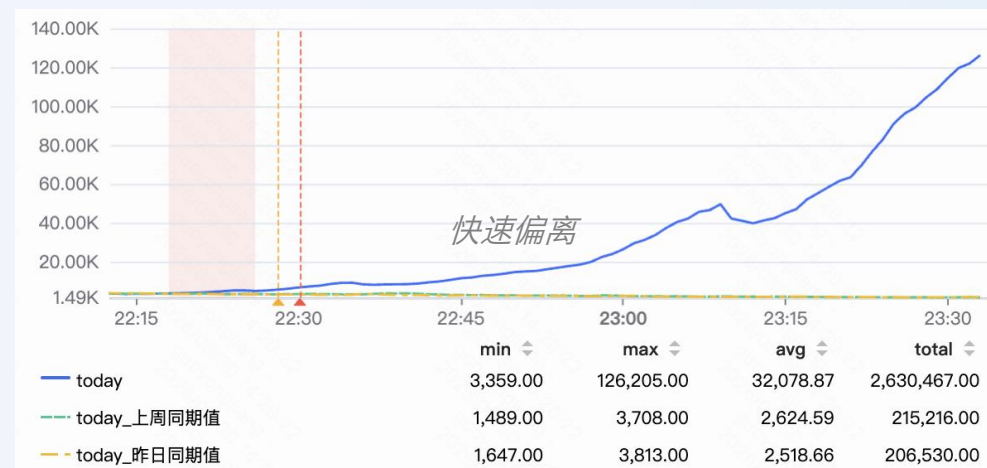
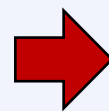


关键洞察：噪声占比 $> 75\%$ ，真正需要关注的告警不足 25%



对抗噪声：告警疲劳的危害

一个P2+case的复盘…



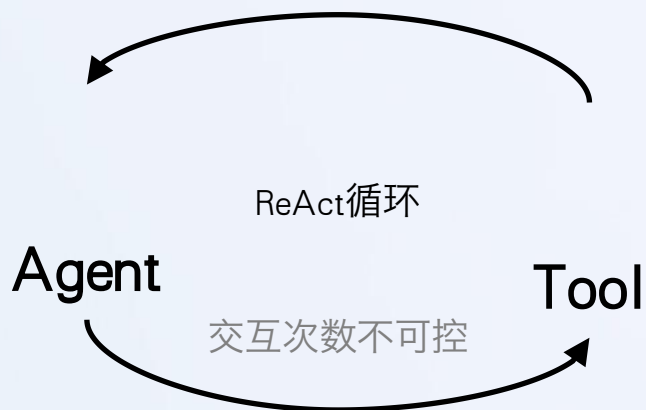
一个指标、不定时波动报警，值班RD接收后点击静默

15min后业务指标迅速偏离，**无感知**

原因：该告警7天内报警次数超过15次，**接麻了**

触发时间	告警内容	当前值	预期值	状态	操作	备注
2025-12-21 22:07:08	出上界，比预期高31.2	已恢复	动态阈值	未静默	天问	
2025-12-18 20:39:09	出上界，比预期高29.2	已恢复	动态阈值	未静默	天问	
2025-12-17 10:57:09	当前值: 8020.00...	已恢复	动态阈值	未静默	天问	
2025-12-17 10:38:09	出上界，比预期高48.8	已恢复	动态阈值	未静默	天问	
2025-12-17 10:14:09	当前值: 3194.00...	已恢复	动态阈值	未静默	天问	
2025-12-16 22:37:08	出上界，比预期高64.8	已恢复	动态阈值	未静默	天问	

对抗噪声：如果让Agent处理全部告警



完整的Agentic推理一次消耗10w~100w Token



快手主站告警事件总量月均2~3w（要求接手率的部分）

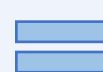
单次深度排查Token消耗



全量告警



月均100亿+Token



年化数百万RMB

Claude sonnet模型计价

除成本不可控外、时间也无法保证

对抗噪声：置信度评估

【入水口】接入归因的告警



告警置信度评估
识别&过滤告警噪声

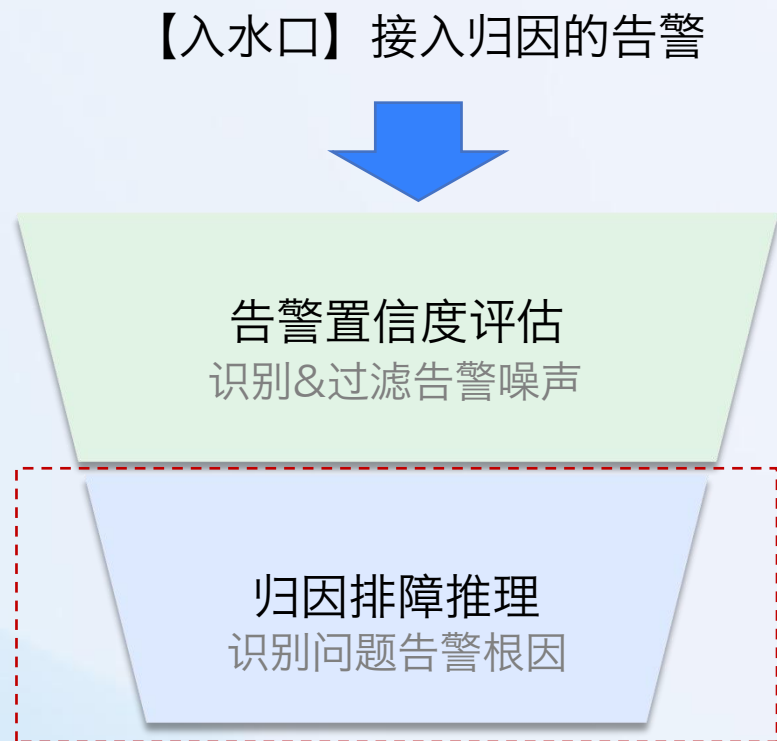
归因排障推理
识别问题告警根因



识别头部告警、重复告警的模式规律，建立告警画像，作为Agent的“外部记忆”

对抗噪声：推理阶段的证据淹没

一些CASE:



过滤误报噪声之后...
推理阶段仍然存在噪声...

指标A波动	→	服务刚好有GC, 但是不足以影响业务	→	误判
指标B波动	→	刚好发布高峰期, 时段内500+关联变更	→	误判
指标C波动	→	下游扇出大有200+, 单服务抖动概率高	→	误判

当系统庞大到一定程度技术指标波动极其频繁。

对抗噪声：证据分级



循证 RCA 流程（引入循证医学思想）



1. 问题发生

定义问题，收集数据



2. 证据检索

工具/ 知识库/ 案例检索



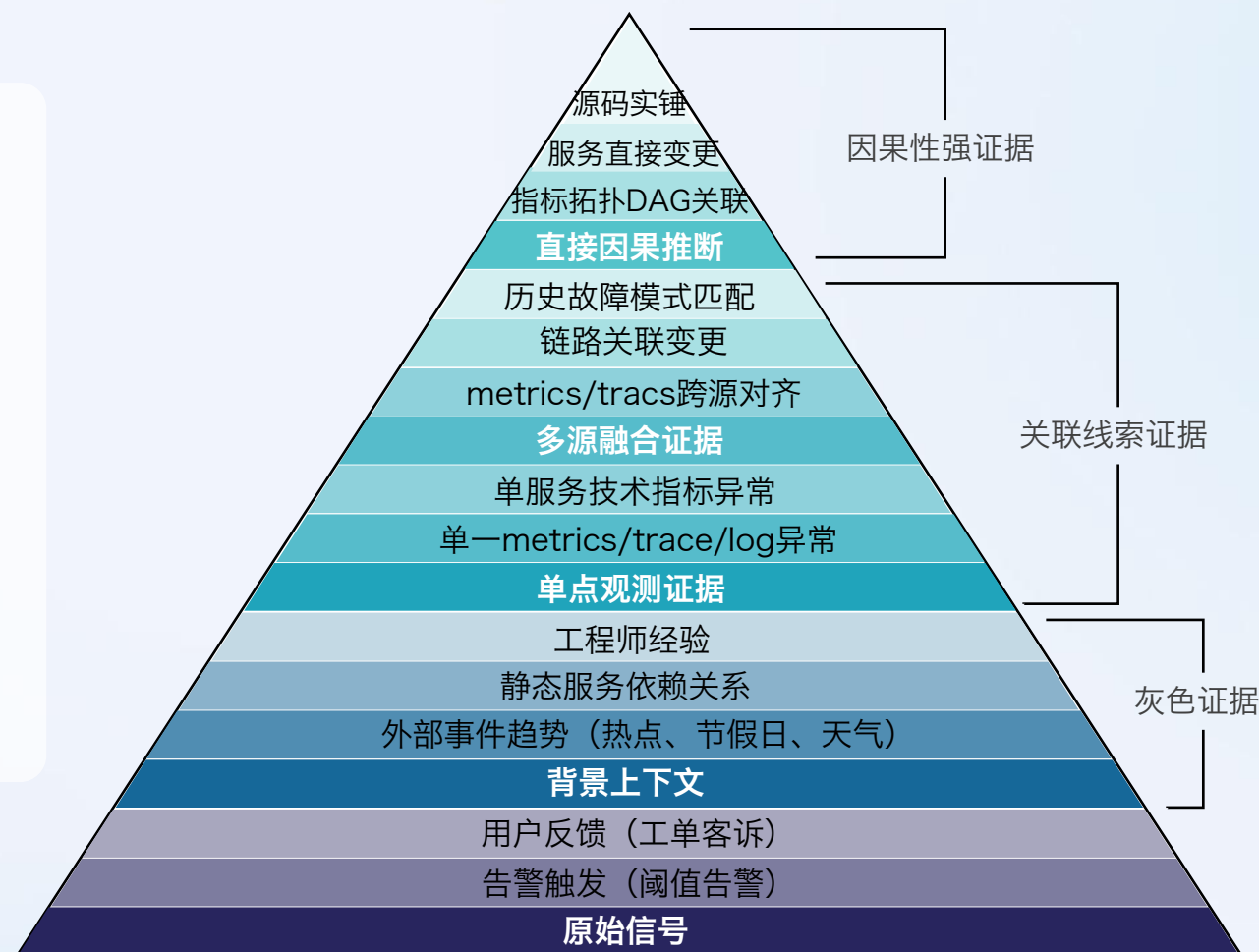
3. 证据评估

可靠性与适用性分级



4. 根因分析

结合最佳证据与现场数据深度分析



层级越高，证据质量与可靠性越强。

挑战三：如何衡量不确定性



生产级agentic系统共识

- 找good case超级容易，比如happy path、cherry-pick例子；
- 彻底消除bad case极其困难，edge cases、silent error等问题层出不穷。



Bob Ng
@bobng2049

PS: bobng是XerpaAI CTO

80% of enterprise AI pilots fail to reach production.

Not because the LLMs are bad. Not because the data is messy.

Because nobody built the orchestration contract before writing a single line of agent code.

After shipping agentic systems in production, I see the same failure pattern repeatedly:

Demo: Agent receives request → calls API → returns clean answer.
Builds in 10 minutes.

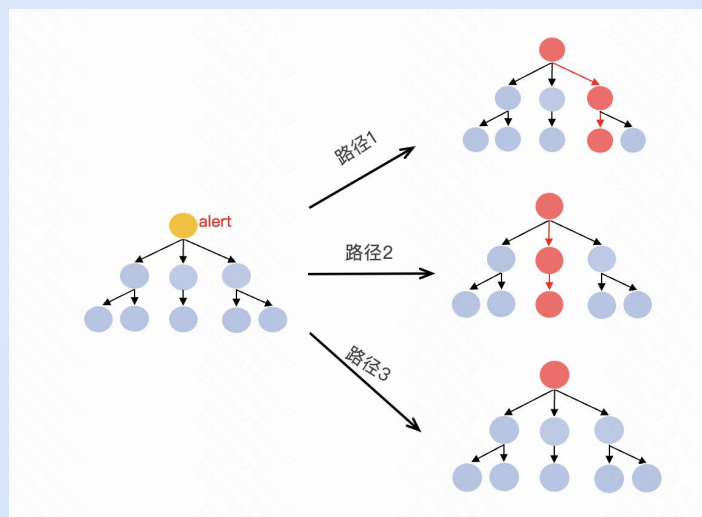
Production reality:

- API returns 503 at 2am
- Customer charged twice on retry
- Compliance requires a full audit trail
- Agent loops on a null response for 6 hours

Demos model happy paths. Production is 90% edge cases.

演示只管顺利的路径，生产90%是坏情况

相比程序的确定性 —— 不确定性是AI常态



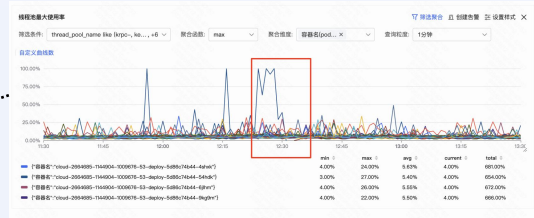
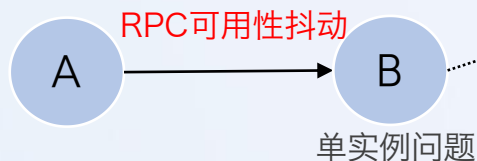
同一个问题输入，多次路径、结论可能不同



类似蝴蝶效应，状态空间的轻微扰动，都会造成结果偏差

衡量不确定性：真实案例 —— 优化变负优化

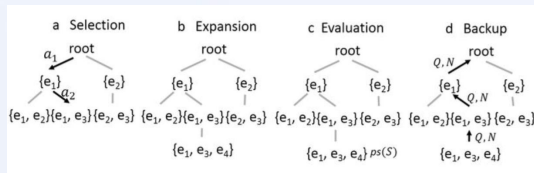
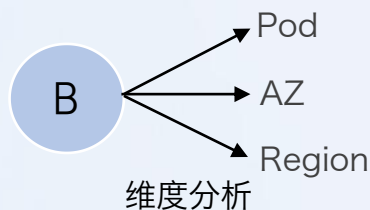
问题：单点抖动



Agent开发的残酷真相

与传统软件的“确定性修复”不同：
你优化了1个bad case，可能又引入了3个新的。

优化：增加【维度分析工具】



引入经典异动分析算法

结果：整体准确度反而劣化

单实例问题发生频率高，导致…

Agent

访问量下降是单实例问题…

送礼金额波动是单实例问题…

搜索量下降是单实例问题…

衡量不确定性：评价体系建立 —— Benchmark平台



Andrej Karpathy ✓

@karpathy

...

sorry just to clarify - the real benchmark of interest is:

"what is the research org agent code that produces improvements on nanochat the fastest?"

this is the new meta.

由 Google 翻译自 英语

抱歉，我再澄清一下——真正重要的衡量标准是：

“哪种研究机构代理代码能够最快地改进 NanoChat？”

这就是新的游戏规则。

评价此翻译：👍 🗨

上午7:35 · 2026年3月6日 · 12.6万 查看

Benchmark是新meta



[Armin Ronacher](#)'s Thoughts and Writings

[blog](#) [archive](#) [projects](#)

Agent Design Is Still Hard

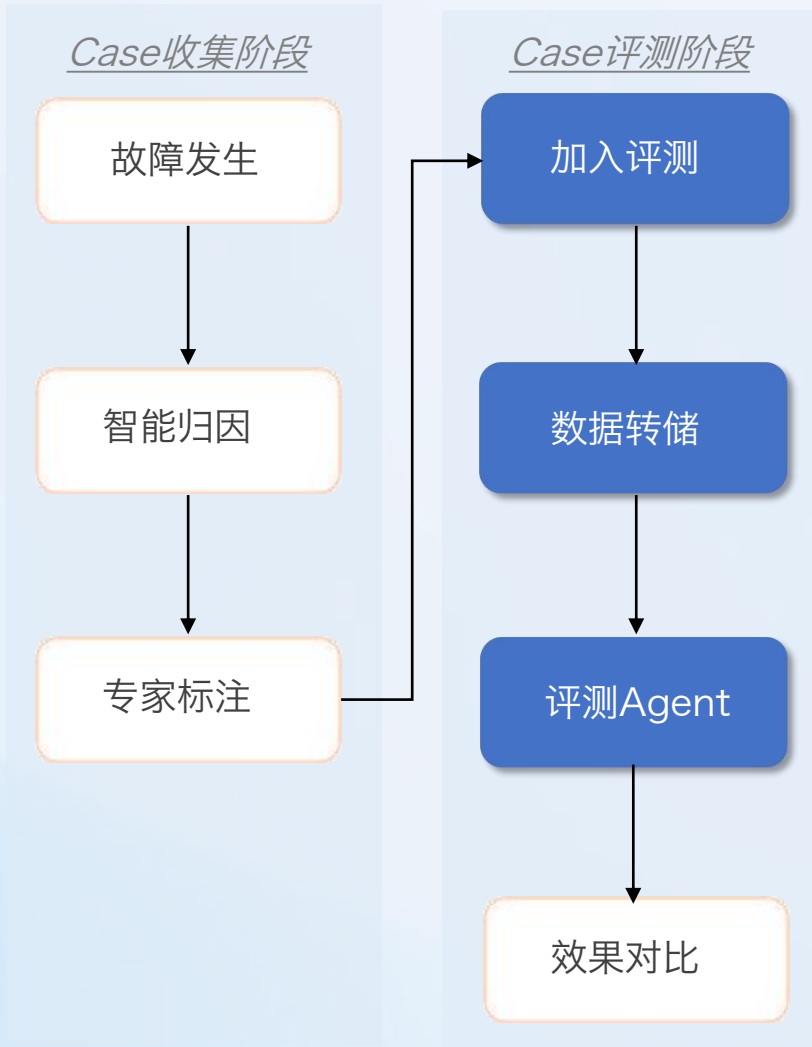
written on November 21, 2025

我们发现测试和评估是这里最难的问题。这并不完全令人惊讶，但智能体的性质使它变得更加困难。与提示不同，你不能只是在某个外部系统中进行评估，因为你需要输入的东西太多了。这意味着你想基于可观察性数据或对实际测试运行进行检测来进行评估。到目前为止，我们尝试过的解决方案都没有让我们相信它们在这里找到了正确的方法。不幸的是，我必须报告，目前我们还没有找到真正让我们满意的东西。我希望我们能找到这个问题的解决方案，因为它正成为构建智能体的一个越来越令人沮丧的方面。

“测试和评估是这里最难的问题”

到一定程度后、必须引入Benchmark。

衡量不确定性：评价体系建立 —— Benchmark平台



评测集设计，目标：覆盖真实问题空间

来源线上真实故障场景，经专家标注后沉淀入库

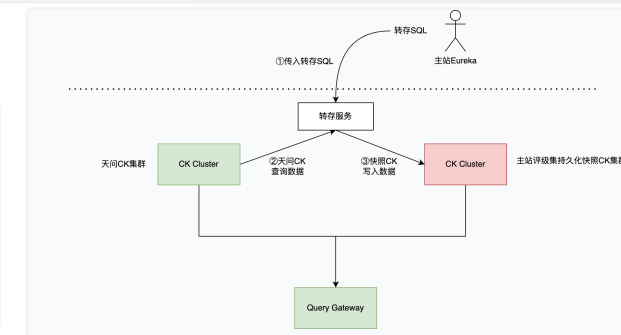
每个任务结构化包含：场景上下文、任务描述、依赖数据、预期结果

The screenshot displays the Benchmark platform interface, showing a list of evaluation tasks and their results. The interface includes a table with columns for task ID, task name, task description, task status, and task results. The table lists several tasks, including '故障发生', '智能归因', and '专家标注'. The results column shows the status of each task, such as '通过' (Pass) or '失败' (Fail).

评测数据构造，目标：复现真实排障环境

"快照式"存储策略：只存储在推理过程中查询过的数据。

快手每天产生巨量监控数据，存在数据过期问题，只存储包含下游链路执行数据



评测结果评估，目标：衡量模型效果

构建自动化评分体系，通过有效线索进行量化评估。

核心指标
线索命中率

输出对比
预期效果评分

The screenshot displays the Benchmark platform interface, showing a list of evaluation tasks and their results. The interface includes a table with columns for task ID, task name, task description, task status, and task results. The table lists several tasks, including '故障发生', '智能归因', and '专家标注'. The results column shows the status of each task, such as '通过' (Pass) or '失败' (Fail).

挑战四：对抗幻觉

一个有趣的CASE PS: 在claude opus4模型下

提示词1（直接询问）

这是中国时间（UTC+8）：2025-11-24 21:07:01
请帮我转换成时间戳

→ 结果几乎完全不准确

提示词2（工具约束）

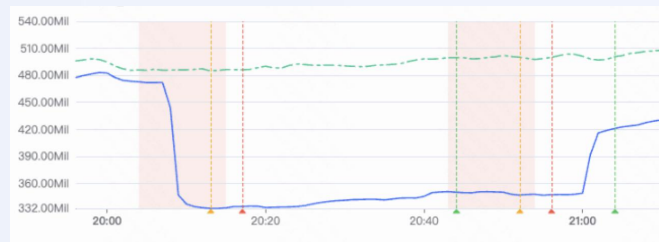
这是中国时间（UTC+8）：2025-11-24 21:07:01
请运行Python脚本帮我转换成时间戳

→ 结果非常精准

关键洞察：LLM是概率预测器，不擅长数值计算类任务

对抗幻觉：用工程提高确定性

实际场景 —— 识别监控图片简单趋势



LLM识别



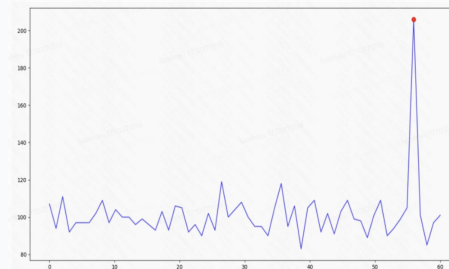
多模态识别：幻觉严重、依赖样式

时序数据识别：耗token、计算出错



传统机器学习算法

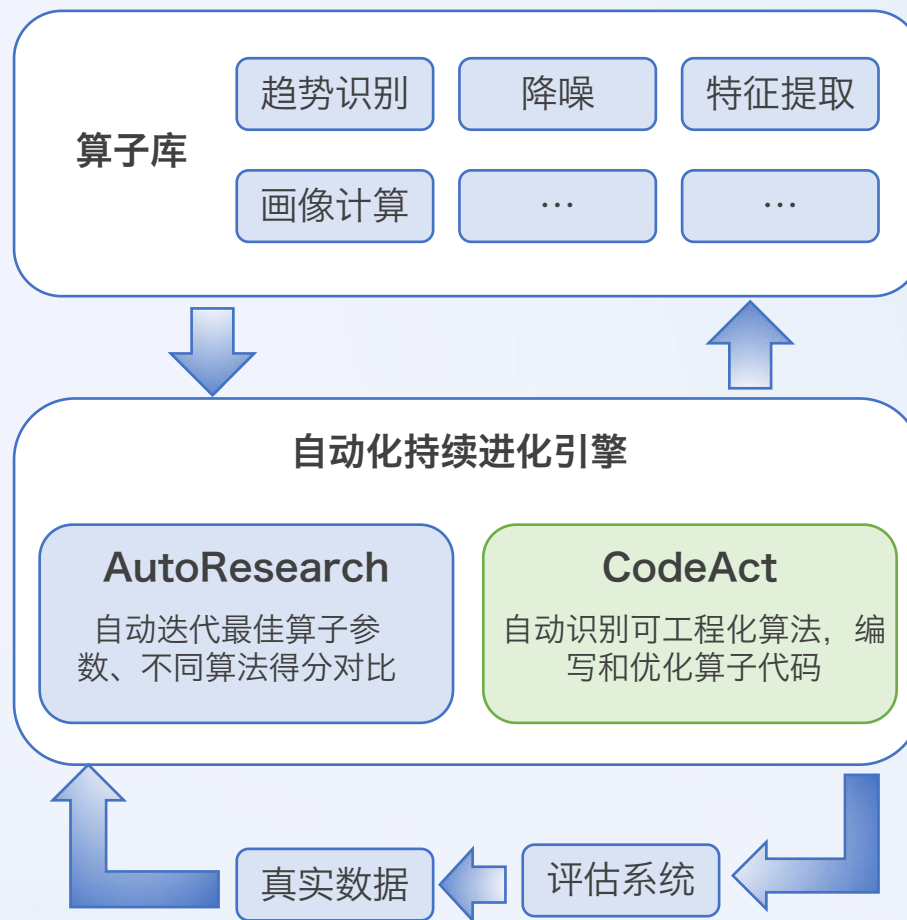
孤立森林算法进行召回+通过k-sigma
算法及切片相似度过滤进行降噪



延迟缩短、Token消耗降低、确定性增强

⚠ 当确定性要求超过一定程度时，工程化封装Tool/Skill是更优解。

对抗幻觉：算子进化



通过算子提升确定性，构建Harness层持续迭代

03

核心机制和架构设计

一些Agent的设计思考&落地形式

演进的路径

AI+归因的发展路线



从Workflow演进到Agent，一定更优吗？

确定性 Workflow

优势 (Pros)

- 稳定可控，执行结果具有高度确定性
- 逻辑白盒化，易于排查、回溯和调试

局限 (Cons)

- 严重依赖预设 SOP，步骤缺乏灵活性
- 仅能覆盖有限且可枚举的已知故障场景
- 维护成本随场景增加呈指数级上升

延迟 ↑
Token ↑



←
确定性 ↑

AI Agent

优势 (Pros)

- 具备自主规划能力，动态决策排障步骤
- 擅长处理发散、跨系统的复杂不完全信息

挑战 (Challenges)

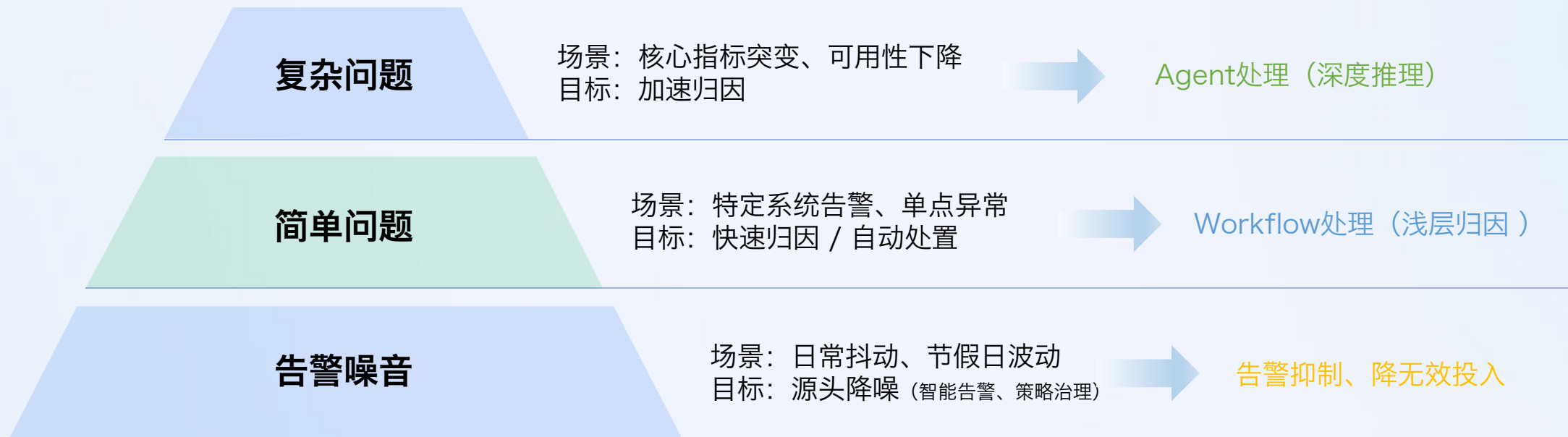
- 大模型存在表现不稳或产生“幻觉”的风险
- 极度依赖完善的工程约束与多维评测保障
- 工具无限膨胀时容易导致推理能力下降



Agent不是“更智能”的代名词，而是“更灵活”的代价

对于跨系统传播、多信号并发、根因高度不确定的开放性问题，无法简单地被静态 SOP 所覆盖，这正是引入 AI Agent 的核心价值所在

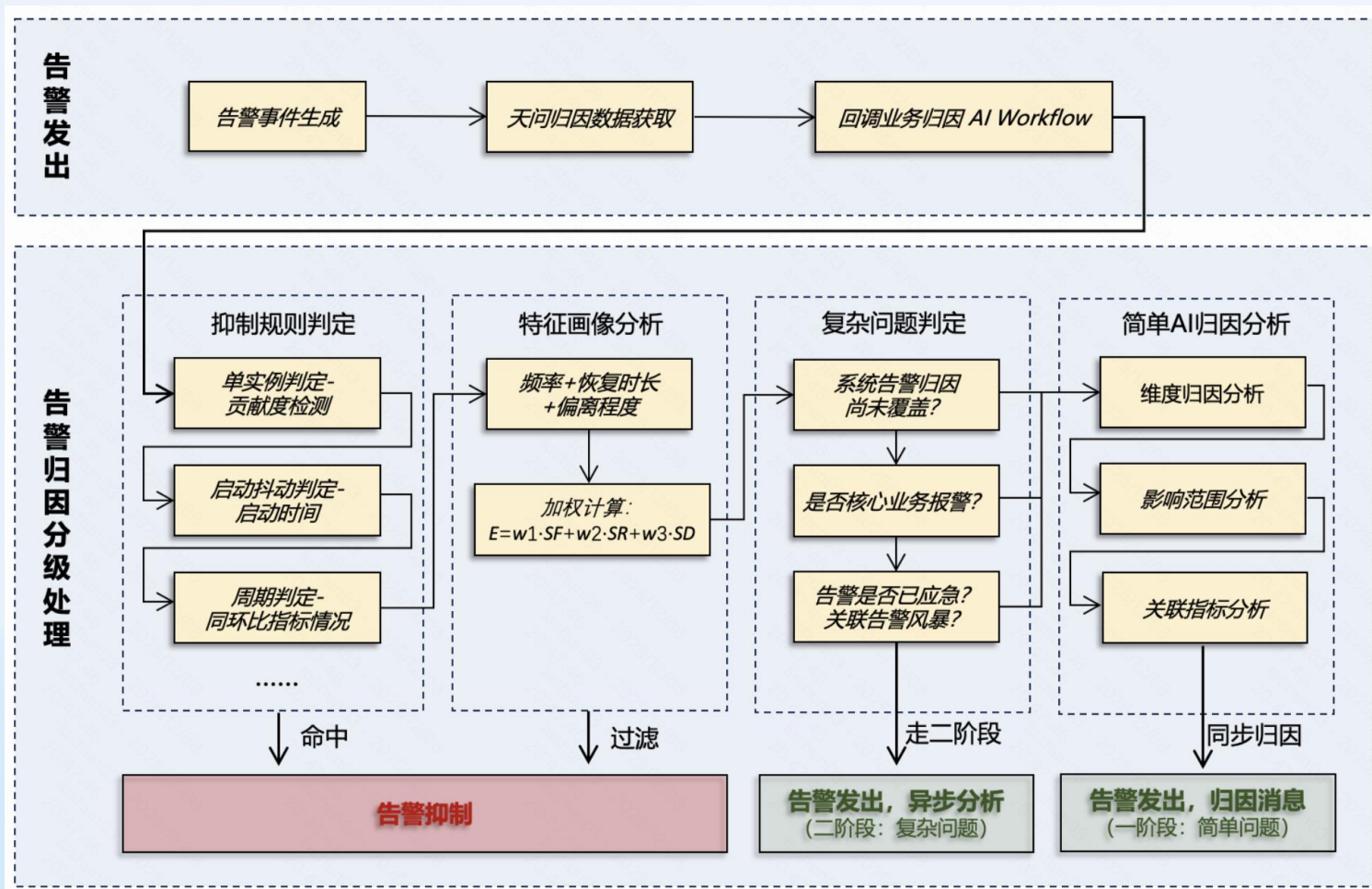
告警分层治理思路



Workflow快思考 + Agent慢思考



快思考 —— 确定性Workflow



Workflow = 确定可控

针对系统类告警:

比如Redis可用性降低、连接数量波动, JAVA异常等问题往往有固定的排查SOP、可枚举的排查步骤。

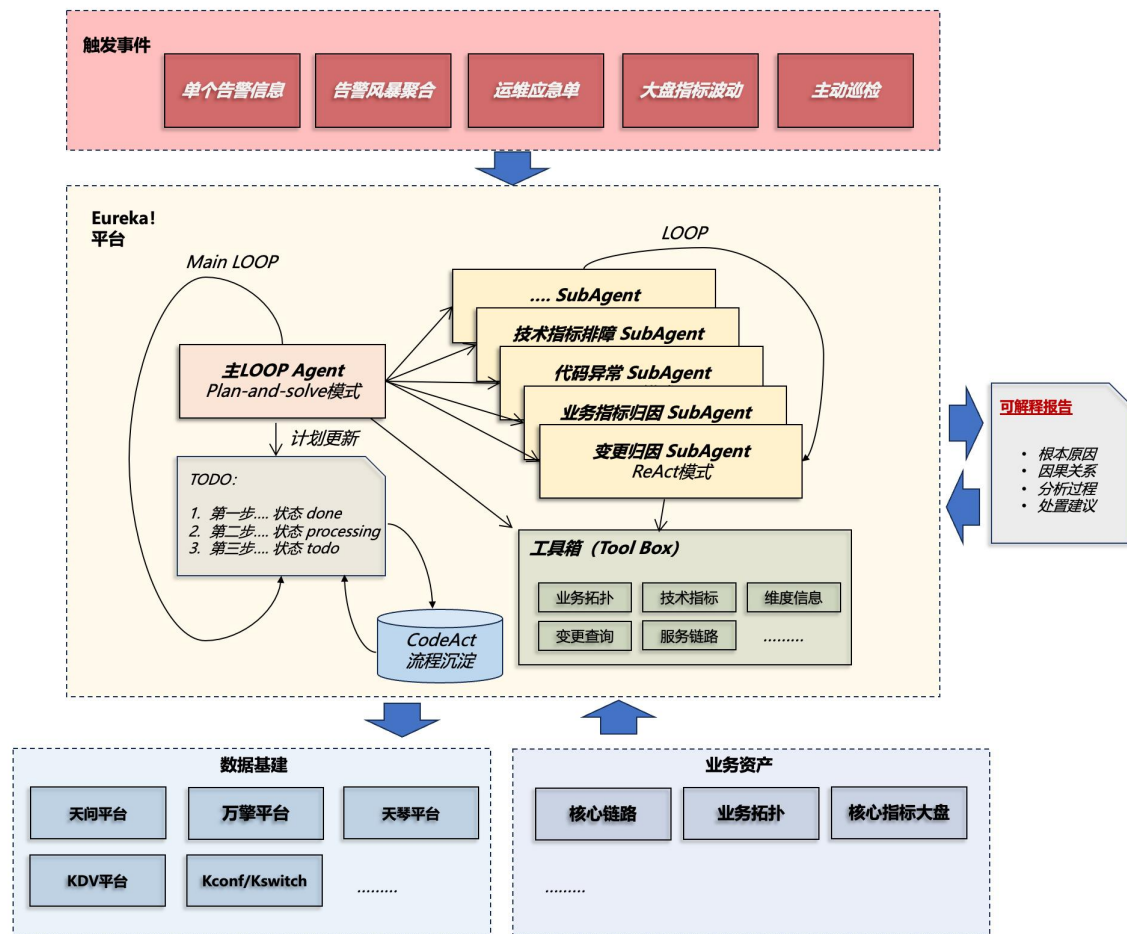
确定性高:

Step By Step的排查流程可以明确步骤, 只需要制定精准的排查流程往往就可能快速取得效果。

快速出结论:

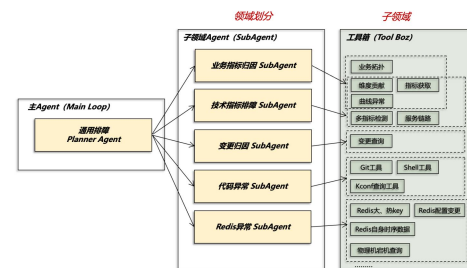
Workflow中和LLM交互次数是可控的, 可以保证在告警发出同时也完成归因分析。

慢思考 —— MultiAgent架构



SubAgent领域封装

类似服务领域拆分，将同领域Tool封装成SubAgent。



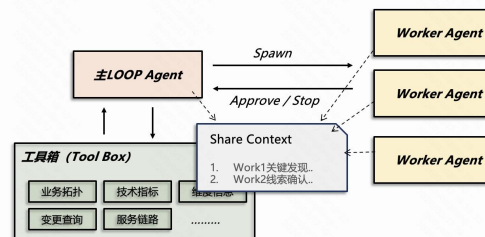
长任务异步执行

比如涉及到要代码分析的，需加快复杂问题排查速度、避免排查阻塞。



Agent Team通信

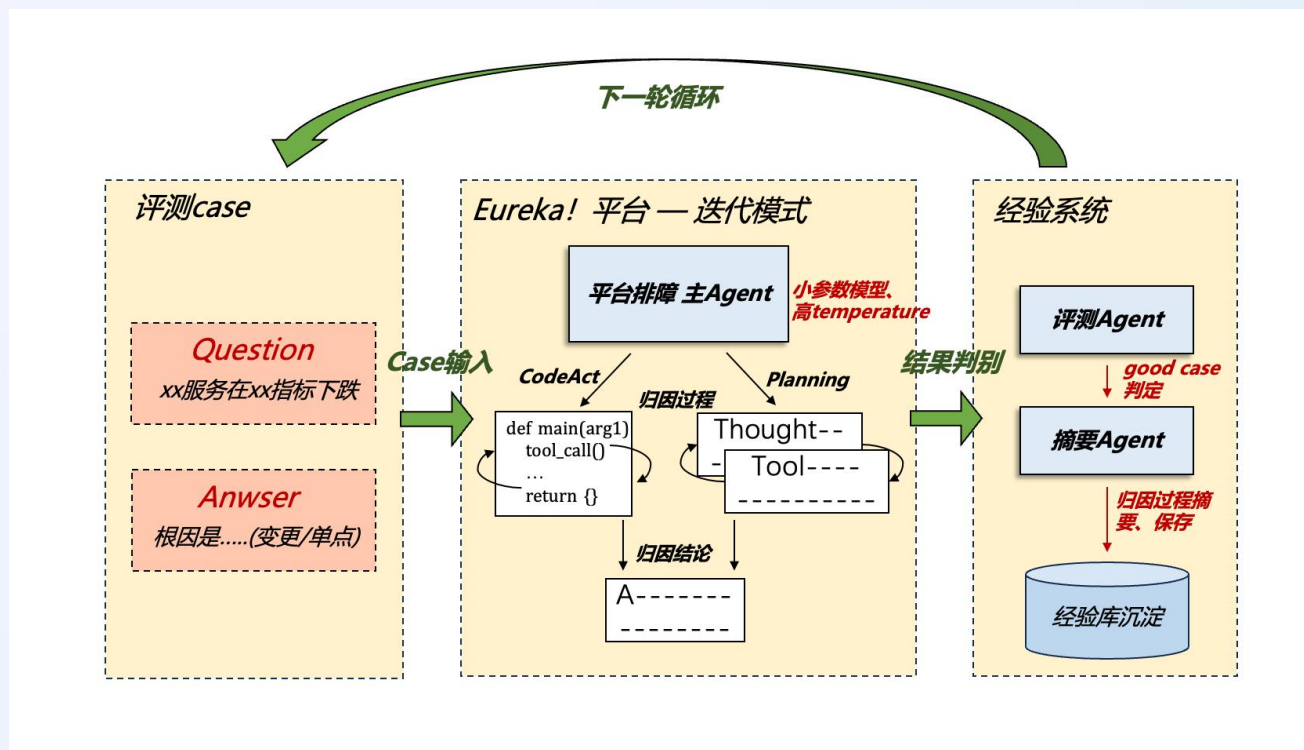
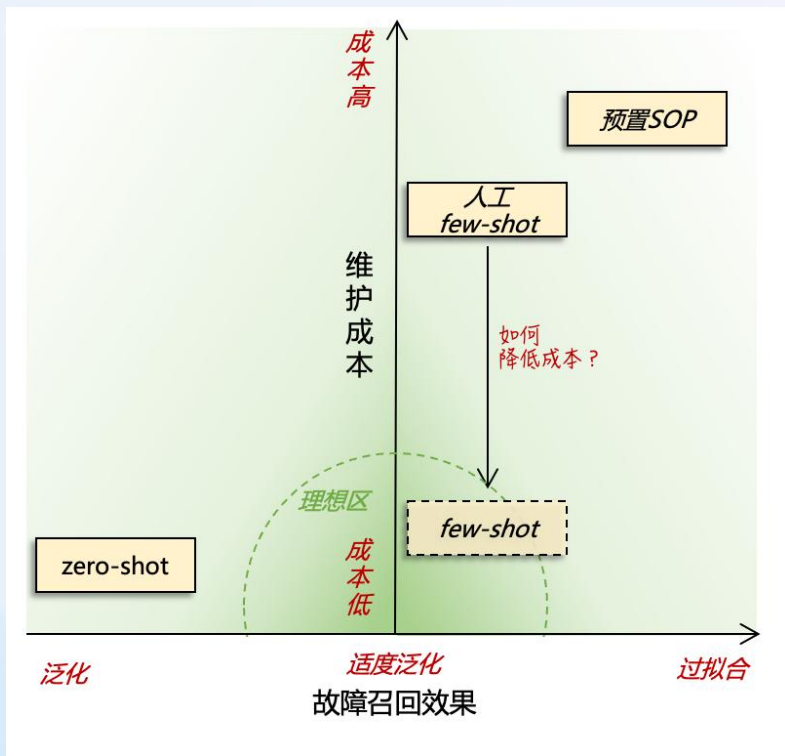
缩短排查路径，避免反复确认。但管理难度增大，需防止上下文污染。



Agent自进化 —— 自动构建案例集

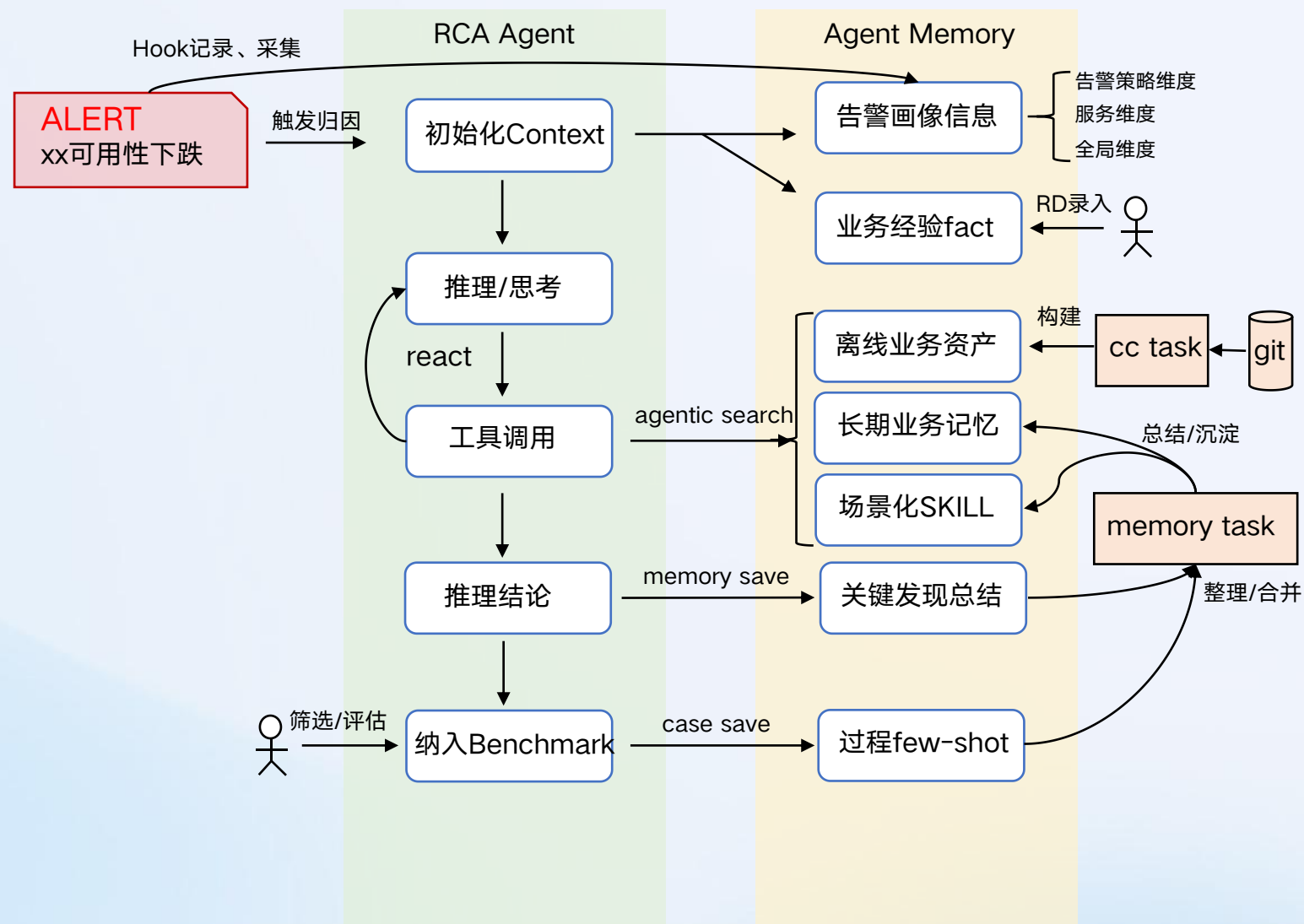
如何低运维成本保证效果？

Agent自动构建案例集/经验库



让模型在当前工具集的条件下进行自我探索，用这种机制将good case的推理路径积累起来。
(积累的few-shot路径，后续同样可以用于RL)

Agent记忆 —— 信息组织



RD脑中的经验-业务事实

"单实例问题很难引发业务指标的波动",

"送礼请求突增可能和大主播口播、直播间送礼弹窗有关",

告警统计数据-历史洞察

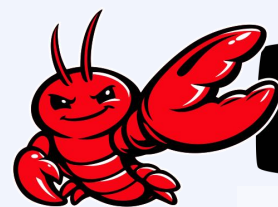
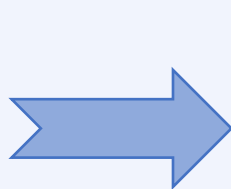
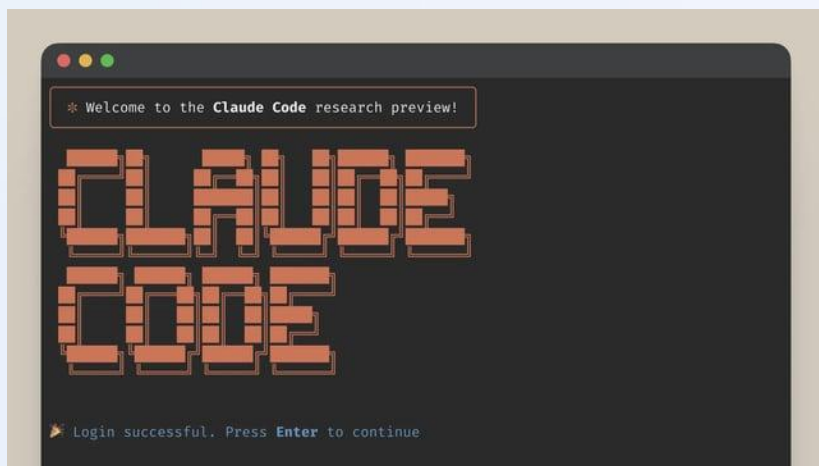
xx告警具有周期性, 每天凌晨1点偏离30%。

xx告警虽然维持一定的告警频率, 但本次告警偏离幅度明显超过历史均值。

Agent历史推理-过往记录

语义匹配, 发现过往曾经推断过x次类似线索的case, 推断原因为xx。

产品交互 —— OpenClaw带来的启示



OPENCLAW



从Claude Code到Open Claw, 增量是什么?

产品交互 —— 从ChatBot到AI Native

从人触发的ChatBot模式，走向AI自主驱动的AI Native模式

输入问题

使用 Eureka! 排查告警/指标/服务.....

请帮我排查告警: lianen-mobile.corp.kuaishou.com/alert/landingpage?msg_id=668ed3891fa911f18c225c6f69c67e70&msg_ts=1773494973&multiple=false&real_source=6&sig=

自动拉群

Agent

【已响应】【260404】主调用下游RP.

8:24 提示:【@应急处置小助手】使用自助查询工

消息 话题

实时推送

以Chatbot的产品形式对外提供能力。面向的场景包括“排障”、“巡检”等。用户在对话框内提问，Agent推理给出完整分析报告。

终态：自动接管并驱动实现“问题感知->归因排障->协同处置->处置建议/分级自动处置->经验沉淀(自学习)”自闭环。

落地成果 —— 核心指标运营

结果指标

缩短故障MTTR

(真正形成的故障样本集过少，可能面临较大波动和不确定性。需要结合过程指标收益综合评估)

- 有效线索率
- 归因准确率
- 归因时长

过程指标

核心关注指标	说明
整体告警 结果准确率 (含拦截、推理结果)	分子 / 分母 = (所有归因推理case准确数 / 所有归因case数量)
全部告警 推准率 (只看进入推理部分)	分子 / 分母 = (推理准确case数 / 所有进入推理case数量)
异常告警 推准率 (异常告警、只看进入推理部分)	分子 / 分母 = (推理准确case数 / 异常告警进入推理case数量)
异常告警 有效线索率 (实际业务异常、进入推理、命中关键线索)	分子 / 分母 = (推理命中正确线索case数 / 所有进入推理case数量)

整体准确率80%+，推理准确率60%+（含线索）

经验沉淀 —— 踩坑和认知



1

LLM层频繁失败

调用不稳定，业务层必须建立Failover机制，确保服务可用性。



2

API 接口不匹配

不同LLM切换后，参数结构与返回格式差异导致API层兼容性问题。



3

模型性能差异

实验结论表明，不同模型在同一任务上的推理能力与耗时存在显著差距。



4

数据缺失处理

当上下文缺失时，需设计降级策略或主动追问机制，而非产生幻觉。



5

重试 vs 立即失败

警惕Agent"钻牛角尖"，如遇到错误服务名，避免自作主张修改导致数据失真，应适时Fail Fast。

本质是软件工程

Agent 应用开发回归本质仍是**软件工程**。核心在于构建健壮的错误处理、故障预防及容错能力体系。

拥抱不确定性

分布式系统本身即在对抗环境不确定性，而 **AI 将进一步放大这种不确定性**，系统设计需从"确定性交付"转向"概率性容错"。

04

总结和展望



从面向人到面向Agent



“拿着旧地图，找不到新大陆”

面向人（现状）

① 上下文有限

只能处理高度抽象、结构化数据。对应着目前的监控曲线、告警规则要足够简单。

② 高度分工

造成经验孤岛。现代软件工程高度分工，人只了解领域内知识，跨领域沟通成本高。

③ 信息即权利

权限隔离和分散的平台信息造成的信息孤岛：谁掌握更多信息，谁就掌握主动权。

面向Agent（新范式）

① 可处理无限的更复杂数据

能够直接消费多模态、多源异构数据，不仅仅局限于简单的指标和日志。

② 上下文同步成本更低

共享记忆机制，链式推理无需人工中转，打破知识孤岛。

③ 透明即效率

Agent取代协调功能，自动获取信息、识别关键服务、生成排查方向。

核心沉淀

"Agent发展非常快，很多两个月前的结论就已过时。我们能积累下来什么？"

可复用的（稳定层）



问题域业务资产

可观测性领域知识与专家经验沉淀



Eval评价体系

完备的测试、评估与体系、故障标注库



结构化案例集

历史经过验证、归因推断成功的过程记录



人机协作模式

明确何时交给人、何时交给Agent

容易被颠覆的（易变层）



具体框架与工具选型

LangChain / Spring AI / AutoGPT 等快速迭代工具



协议与规范

随着SKILL和CLI的流行，MCP 逐渐被抛弃



Prompt描述

随业务迭代、模型能力提升而变化的提示词工程



模型选型

基座模型能力的快速更迭

未来演进

向更加 AI Native自主化、AI自进化的方向演进

1

辅助决策

人工主导

AI提供建议与洞察，辅助人进行可观测数据的智能分析。

角色：AI作为副驾驶 (Copilot)

2

自主执行

Agent执行，人审批

Agent自主执行告警分析、根因定位、自动修复，人负责关键节点的审批与监督。

角色：AI作为代理人 (Agent)

3

自我进化

AI自主闭环

AI自主优化自身的观测策略、告警规则和修复手段，持续学习进化，无需人工干预。

角色：AI作为自主系统 (Autonomous)

关键挑战

🔍 可解释性

🛡️ 安全边界

🤝 人机信任建立

QCon
全球软件开发大会

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The
Software



G.YL

广东 深圳



扫一扫上面的二维码图案，加我为朋友。

AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

